새싹 학습 노트 · 2026.11.02 · Day11 · CNN·이미지 분류·전이학습 · 수강생용 개념 지도

CNN 개념 지도 · 모델 안과 모델 밖

오늘 배우는 개념은 두 장의 지도로 나뉩니다. 배우는 범위는 강의 PDF, 개념의 정의와 관계는 **UDL**(Understanding Deep Learning)을 따랐습니다.

지도 A · 모델 안
사진 한 장이 점수가 되기까지. 한 칸씩 확대해 Kernel까지

지도 B · 모델 밖
데이터 준비부터 성능표까지, 모델을 만들고 고르는 작업 순서

BEFORE THE MAPS

지도를 읽기 전에 · 기초 개념

지도에 설명 없이 나오는 말들입니다. 배지는 그 말이 나오는 지도 위치입니다.

이미 배운 것

Day10까지의 내용. 지도에 그대로 다시 나옵니다.

Tensor와 Shape Tensor · Shape
숫자를 담은 다차원 상자. 이미지 batch는 `[N, C, H, W]` = 이미지 수 · 채널 · 높이 · 너비.
A-① UDL 부록 B.3

뉴런 = 가중합 + bias hidden unit
입력마다 가중치를 곱해 더하고 bias를 더함. Linear도, Conv도 이 계산을 합니다.
A-① Head A-③ UDL 3.1 · 10.2.4

활성화 함수 ReLU activation
음수는 0, 양수는 그대로. 이런 비선형이 있어야 층을 쌓는 의미가 생깁니다.
A-② UDL 3.1

파라미터 vs 하이퍼파라미터 parameter · hyperparameter
파라미터는 학습으로 정해지는 값(Kernel 안의 숫자). **하이퍼파라미터**는 사람이 정하는 값(Kernel 크기, Stride, lr).
A-③ B-2 UDL 8.5

Logit → Softmax → 손실 logit · softmax · cross-entropy
Head가 낸 점수(logit)를 확률로 바꾸고, 정답 확률이 낮을수록 손실이 큼. 학습은 이 손실을 줄이도록 가중치를 고치는 일 (Day10).
A-① Head B-3 UDL 5.5 · 6장

train / valid / test와 과적합 overfitting
배우기 · 고르기 · 최종 확인용 데이터. train만 잘 맞히고 새 데이터에 약하면 과적합. 증강·전이학습은 이를 줄이는 방법.
B-1 · B-4 · B-6 UDL 8장 · 9.3

오늘 새로 나오는 것

Day11의 핵심 용어. 위쪽 파란 선으로 구분했습니다.

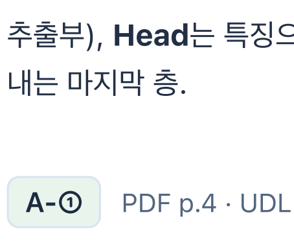
합성곱 convolution
작은 창을 옮겨 가며, 창 안의 값을 가중합하는 연산. 모든 위치에서 같은 가중치를 씁니다.
A-③ UDL 10.2.1

Kernel (필터) kernel · filter
합성곱 창 안의 가중치 묶음. 3×3이면 9개 숫자이고, 학습으로 정해지는 **파라미터**입니다.
A-③ UDL 10.2.1

Feature Map feature map
Kernel 하나가 이미지 전체를 훑어 만든 결과 지도. "특징 추출부"의 특징이 바로 이것입니다.
A-② Conv2d UDL 10.2.5

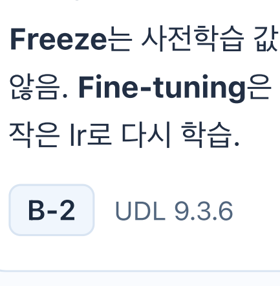
채널 channel
Feature Map 한 장 = 채널 하나. 입력 RGB는 3채널, 출력 채널 수 = Kernel 개수.
A-③ UDL 10.2.5 · 10.3

Stride stride
창이 한 번에 건너뛰는 칸 수. 클수록 출력이 작아짐.



칠한 칸 = 창(3칸)이 시작하는 위치. stride 2면 출력이 약 절반
A-③ UDL 10.2.3

Padding padding
가장자리 바깥에 값(보통 0)을 덧대기. 덧대지 않으면(valid) 출력이 줄어듦.



4×4 입력 둘레에 0을 한 겹(P=1)
→ 3×3 창을 써도 출력이 4×4로 유지
A-③ UDL 10.2.2

Pooling pooling · downsampling
작은 영역을 대표값 하나로 줄이기. 채널마다 따로 하므로 채널 수는 그대로.

$$\begin{bmatrix} 1 & \boxed{4} \\ 2 & 3 \end{bmatrix} \rightarrow \boxed{4}$$

2×2 MaxPool: 가장 큰 값 하나 (PDF p.5)
A-② MaxPool UDL 10.4.1

수용영역 receptive field
출력 한 칸이 원본 이미지에서 보는 범위. 층을 쌓고 Pooling을 거칠수록 넓어짐.
A-② A-① UDL 10.2.6

Backbone과 Head backbone · head
Backbone은 특징을 만드는 앞부분(특징 추출부), **Head**는 특징으로 클래스 점수를 내는 마지막 층.
A-① PDF p.4 · UDL 9.3.6

Freeze와 Fine-tuning freeze · fine-tuning
Freeze는 사전학습 값을 고정해 학습하지 않음. **Fine-tuning**은 일부 또는 전체를 작은 lr로 다시 학습.
B-2 UDL 9.3.6

MAP A · INSIDE THE MODEL

지도 A · 모델 안 — 사진 한 장이 점수가 되기까지

읽는 법 → **오른쪽**으로 갈수록 사진이 점수로 바뀝니다 ▼ **아래**로 갈수록 한 칸을 확대해 봅니다

1 CNN 모델 전체 · 한 줄 흐름 SmallCNN(PDF p.41)에 64×64 사진을 넣은 경우

사진 (이미지 Tensor) N장 · RGB 3채널 · 64×64 `[N,3,64,64]`
→ **특징 추출부 = Conv 블록 × 2** 블록마다 공간은 ½, 채널은 늘어남 `[N,32,16,16]` ▼ ②에서 블록 하나를 확대
→ **Global Avg Pool** 채널마다 평균 하나. 위치 정보를 버림 `[N,32,1,1]`
→ **flatten(x, 1)** 벡터로 펴기. N축은 남김 `[N,32]`
→ **Head (Linear 32→K)** 클래스별 점수(logit) `[N,K]`

Backbone · 전이학습에서는 사전학습 값을 재사용
Head · K에 맞게 교체

공간 64 → 16 → 1, 채널 3 → 32. "어디에 있나"를 점점 버리고 "무엇이 있나"로 바꾸는 과정입니다. 큰 모델(VGG·ResNet)도 같은 모양입니다.
UDL 10.5.1 · 10.6

Conv는 사진을 옮기면 결과도 같이 옮겨지고(등변), Pool은 작은 이동을 흡수합니다(불변). 그래서 물체가 사진의 어디에 있어도 비슷한 점수가 나옵니다.
UDL 10.1 · 10.4.1

▼ 특징 추출부 안의 Conv 블록 하나를 확대

2 Conv 블록 하나 예: 첫 블록, 입력 `[8,3,64,64]` (PDF p.5)

Conv2d 작은 창으로 패턴을 찾아 Feature Map을 만듦 `[8,16,64,64]` ▼ ③에서 확대
→ **BatchNorm2d** 채널마다 값의 평균·크기를 맞춤. train/eval 동작이 다름 `Shape 그대로` SmallCNN엔 없고 ResNet18엔 있음
→ **ReLU** 음수를 0으로 바꾸는 비선형 `[8,16,64,64]`
→ **MaxPool2d(2)** 2×2에서 최댓값 하나. 채널 수는 그대로 `[8,16,32,32]`

MaxPool로 공간을 ½ → 다음 Conv의 한 칸이 원본의 더 넓은 범위를 봅니다. 한 칸이 보는 범위(수용영역): 3 → 4 → 8 → 10 → Global Pool에서 전체.
UDL 10.2.6

ReLU가 없으면 층을 아무리 쌓아도 선형 하나와 같습니다. 블록을 반복하는 의미를 만드는 부분입니다.
UDL 3장 · 4장

BatchNorm은 깊은 망의 값이 층마다 커지는 것을 막아 큰 학습률에도 학습이 안정됩니다. 평가 때는 저장된 통계를 씁니다.
UDL 11.4

▼ Conv2d 안의 Kernel 연산을 확대

3 Kernel 연산 작은 창의 값과 가중치를 곱해 합산 → Feature Map의 한 칸

$$\underbrace{\begin{bmatrix} \boxed{0} & \boxed{1} & \boxed{0} & 0 & 0 \\ \boxed{0} & \boxed{1} & \boxed{0} & 0 & 0 \\ \boxed{0} & \boxed{1} & \boxed{0} & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}}_{\text{입력 5×5}} \times \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}}_{\text{Kernel 3×3}} = \underbrace{\begin{bmatrix} \boxed{3} & 0 & 0 \\ 3 & 0 & 0 \\ 3 & 0 & 0 \end{bmatrix}}_{\text{Feature Map 3×3}}$$

강조한 영역 × Kernel = **1 + 1 + 1 = 3** → 출력 첫 칸. 창을 한 칸씩 옮기며 반복합니다. 이 Kernel은 세로선에 반응합니다. 실제 Kernel 값은 학습으로 정해집니다. (PDF p.6 · stride 1, padding 0, bias 0)

한 칸의 계산 $h_{ij} = a[\beta + \sum\sum \omega_{mn} \cdot x_{i+m-2,\ j+n-2}]$ (UDL 식 10.6)
출력 크기 $H_{out} = \lfloor (H + 2P - D(K-1) - 1) / S \rfloor + 1$ (PDF p.26)

Kernel 크기 K
창의 크기. 가운데를 두려고 보통 홀수

Stride S
창이 건너뛰는 칸 수. 2면 출력이 약 ½

Padding P
가장자리에 0을 덧댐. 3×3에 P=1이면 크기 유지

채널 C_{in} → C_{out}
Kernel 하나가 출력 채널 하나. RGB면 Kernel 하나가 3채널을 모두 봄

같은 Kernel을 모든 위치에 씁니다(가중치 공유). Conv 3→16(3×3)은 3·16·3·3 + 16 = 448개입니다. 같은 사진을 펼쳐 뉴런 128개에 연결하면 첫 층만 약 157만 개입니다. 파라미터 수는 사진 크기와 무관합니다.
UDL 10.2.4 · 10.3 · PDF p.18

근거: UDL 10.1 등변·불변 · 10.2 합성곱(Kernel·Stride·Padding·채널·수용영역) · 10.3 2D 합성곱 · 10.4 Downsampling · 10.6 요약 · 11.4 BatchNorm / 강의 PDF p.4~6 · 26 · 39 · 41

MAP B · AROUND THE MODEL

지도 B · 모델 밖 — 모델을 만들고 고르는 작업 순서

읽는 법 → **오른쪽**으로 갈수록 작업 순서입니다 단계마다 **배우는 교시**와 **지도 A와 만나는 곳**을 적었습니다

1 데이터 준비

4교시

- **ImageFolder**: 폴더 이름이 곧 라벨
- **Transforms**: 크기 맞추기·Normalize
- **증강은 train만**: 뒤집기·회전처럼 라벨이 안 바뀌는 변환으로 데이터를 늘림 (UDL 9.3.8)
- **DataLoader**: batch로 묶기

지도 A ① 입력 `[N,3,H,W]`을 만듦

→

2 모델 후보 정하기

3·6교시

- **SmallCNN**: 처음부터 학습
- **ResNet18 Head-only**: Head만 학습
- **ResNet18 Partial**: layer4 + Head 학습

지도 A Backbone·Head 중 어디를 학습할지 ▼ 아래 표

→

3 학습

5교시

- **CrossEntropyLoss**: 점수 → Softmax 확률 → 정답 확률의 −log (UDL 5.5)
- **학습 루프**: Day10과 같음. forward → loss → backward → step

지도 A ① 끝의 점수 `[N,K]`를 받음

4 평가 · 저장

5교시

- **eval() + inference_mode**로 valid 평가
- **Best model**: valid 기준 저장, 클래스 순서·Normalize도 함께
- **오류 분석**: Confusion Matrix·클래스별 Recall

지도 A ② BatchNorm이 저장된 통계로 바뀜

→

5 장치 비교

7교시

- **같은 조건**: 데이터·batch·모델·epoch
- **시간**: warm-up 후 synchronize 전후로
- **images/s · Peak VRAM** 기록
- GPU가 없으면 N/A와 사유

지도 A 모델은 그대로, 계산 장소만 바뀜

→

6 성능표 · 설명

8교시

- **성능표**: 모델·학습 범위·valid 지표·시간·VRAM
- **README**: 어떤 모델을 왜 골랐나
- **test**는 모든 선택이 끝난 뒤 한 번

지도 A 어떤 모델(①의 구성)을 골랐는지 기록

▼ 2 · 모델 후보를 확대 — 전이학습: 어디까지 얼리고 어디부터 배우나

전이학습: 데이터가 많은 관련 작업으로 미리 학습한 모델에서 마지막 층을 떼고 새 층을 붙입니다. 본체를 고정하고 새 층만 학습하거나, 전체를 미세조정합니다. 파라미터 대부분을 **좋은 출발점에서 시작**하게 하는 방법으로도 볼 수 있습니다.
UDL 9.3.6

ResNet18 구간 → `224×224 입력 기준`	conv1 + pool `64ch · 56×56`	layer1 `64ch · 56×56`	layer2 `128ch · 28×28`	layer3 `256ch · 14×14`	layer4 `512ch · 7×7`	fc (Head) `512 → K`
처음부터 학습 예: SmallCNN	무작위	무작위	무작위	무작위	무작위	무작위
Head-only PDF p.75	고정	고정	고정	고정	고정	학습 `lr 1e-3`
부분 미세조정 Partial · PDF p.77	고정	고정	고정	고정	학습 `lr 1e-4`	학습 `lr 1e-3`
전체 미세조정 UDL 9.3.6 · 성능표 비교 밖	학습 `작은 lr`	학습	학습	학습	학습	학습

▨ 고정 (사전학습 값 유지) ■ 학습 · 작은 lr ■ 학습 · 큰 lr (새 Head) ☐ 무작위 초기값에서 학습

뒤쪽부터 엽니다. 앞쪽 층은 색·질감처럼 일반적인 특징, 뒤쪽 층은 물체 종류와 더 관련됩니다.
UDL 10장 Notes (Bau et al., 2017)

고정한 층의 BatchNorm은 eval()로 둡니다. `model.train()`을 다시 부르면 저장된 통계가 바뀝니다.
PDF p.75·77 · UDL 11.4

사전학습 때와 같은 입력 조건 (`weights.transforms()`)을 쓰고, fc는 클래스 수 K에 맞게 교체합니다.
PDF p.74·79

근거: UDL 5.5 다중 분류 손실 · 9.3.6 전이학습 · 9.3.8 증강 / 강의 PDF p.7~11 · 54 · 61 · 65~67 · 75~77 · 84 · 89. ImageFolder와 CPU·GPU 측정은 UDL에서 다루지 않아 강의 PDF 기준입니다.

SAME MAPS, IN CODE

같은 지도를 코드로 보기

주석의 A·B가 지도 위치입니다. 지도 A는 모델 클래스 안, 지도 B는 클래스 밖입니다.

Python · 설명용 골격 (실습 원문 아님, PDF p.41·75 기반)

```python
# B-1 데이터 준비 ─────────────────────────────────────────────
train_ds = ImageFolder("data/train", transform=train_tf)   # 증강은 train에만
valid_ds = ImageFolder("data/valid", transform=valid_tf)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)

# 지도 A · 모델 안 ────────────────────────────────────────────
class SmallCNN(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.features = nn.Sequential(       # A-① 특징 추출부
            nn.Conv2d(3, 16, 3, padding=1),  # A-② Conv 블록 (A-③ Kernel은 이 안)
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(16, 32, 3, padding=1), # A-② Conv 블록 두 번째
            nn.ReLU(),
            nn.MaxPool2d(2),
        )
        self.pool = nn.AdaptiveAvgPool2d(1)  # A-① Global Avg Pool
        self.fc = nn.Linear(32, num_classes) # A-① Head

    def forward(self, x):                    # [N,3,H,W]
        x = self.features(x)                 # [N,32,H/4,W/4]
        x = torch.flatten(self.pool(x), 1)   # [N,32]
        return self.fc(x)                    # [N,K] 점수

# B-2 모델 후보 · 전이학습 ────────────────────────────────────
model = resnet18(weights=ResNet18_Weights.DEFAULT)
for p in model.parameters():
    p.requires_grad = False                  # Backbone 고정
model.fc = nn.Linear(model.fc.in_features, num_classes)  # Head 교체

# B-3 학습 ~ B-6 성능표 ───────────────────────────────────────
loss_fn = nn.CrossEntropyLoss()              # Softmax 포함
# 학습 루프 · best model 저장 = Day10과 같음
```

새싹 NVIDIA AI 코어 엔지니어 2기 · Day11 학습 노트 · 2026.11.02 · 개념 기준: Prince, *Understanding Deep Learning* (MIT Press, 2023)